Study book

Topic 2: Collections + LINQ

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 1, topic 2 of 20.

Brief overview

What this lesson is about. The C# collections that replace JS arrays, Maps and Sets (`List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>`, `IEnumerable<T>`), lambdas and delegate types, and **LINQ**: the C# version of `filter` / `map` / `find` / `reduce`. The most important idea is **deferred execution**: a LINQ query is a recipe that runs when it is read, and runs again every time it is read.
Why it matters for the interview. LINQ is everywhere in .NET backends, and EF Core (topics 10-11) uses the same LINQ to generate SQL. Deferred execution, `First` vs `Single`, `IEnumerable` vs `IQueryable`, and 'why Dictionary' are standard interview questions.
What we built. `SampleData.cs` (3 projects, 7 tasks) and a 13-section runnable demo, `playground/02-linq.cs`. You ran a debugger session in Rider that showed deferred execution live.
How to use this book. Read it once fully; later use Part 10 (Q&A) and Part 11 (summary + cheat sheet) to refresh.

Contents

Part	What is inside
1	Collections: List, Dictionary, HashSet, arrays, IEnumerable, and which type to use for parameters
2	Lambdas and delegate types (Func, Action)
3	What LINQ is: extension methods (and the `this` keyword)
4	The JS-to-LINQ translation table, the find family, method vs query syntax, anonymous types
5	Deferred execution, ToList, multiple enumeration, IEnumerable vs IQueryable
6	Gotchas: where C# throws and JS does not
7	`var`: when it is required, when a type is required, and style (your question)
8	Our code: SampleData.cs and the demo file explained, how to open and run it, full output
9	Debugging in Rider: watching deferred execution live, the call stack (Frames), an exception
10	Your questions, mistakes we corrected, interview Q&A with model answers
11	Summary, cheat sheet, checklist, what comes next

Part 1. Collections

C#	What it is	TS / JS analogy
`T[]` (array)	fixed size, fast index access; size via `Length`	a `T[]` you never push to
`List<T>`	growable list: `Add`, `Remove`, `Count`, `[i]`. The everyday collection	`Array<T>`
`Dictionary<TKey, TValue>`	key to value lookup in O(1) (hash table)	`Map<K, V>`
`HashSet<T>`	unique values, fast `Contains`	`Set<T>`
`IEnumerable<T>`	'something you can loop over'. Read-only, possibly lazy	`Iterable<T>`
`IReadOnlyList<T>` / `IReadOnlyCollection<T>`	read-only view with `Count` (and index)	`ReadonlyArray<T>`

Differences from JS worth knowing

- `List<T>` has `Count`, not `length`. Arrays have `Length`. (Inconsistent, but that is .NET.)
- `List<T>` holds only `T`: you cannot push a string into `List<int>`. Generics are checked at compile time and at runtime.
- `dict[key]` **throws** `KeyNotFoundException` when the key is missing (JS `map.get` returns `undefined`). Use `TryGetValue` when a key may be missing.
- `list.Sort()` sorts **in place**, like JS `sort`. LINQ `OrderBy` returns a **new** (lazy) sequence and leaves the source untouched, like `toSorted` or lodash `orderBy`.

Which type for a method parameter? (CHECK question 2)

`IEnumerable<T>` is an **interface** (topic 3). `List<T>`, arrays, `HashSet<T>` and others implement it, so a method that takes `IEnumerable<TaskItem>` accepts all of them, like a TS function taking `Iterable<T>`. Rule: **accept the most general type you need, return the most specific useful type.**

Your answer

Your answer: 'IEnumerable, because List is only List; a method taking IEnumerable is more reusable.'
Correct.
Refinements an interviewer may push on: (1) an `IEnumerable` may be a lazy query, so if the method loops over it twice the work runs twice; (2) if the method needs `.Count` or an index, take `IReadOnlyCollection<T>` / `IReadOnlyList<T>` instead: still general, but guaranteed materialized.

Dictionary vs List for lookups (CHECK question 3)

You answered 'Dictionary of course'. Right choice, but the interview needs the **reason**:

- `Dictionary` is a hash table: a lookup jumps straight to the key, **O(1)**.
- `List` + `FirstOrDefault` scans item by item, **O(n)**. 1,000 lookups in 10,000 projects: up to 10 million comparisons vs about 1,000.
- Trade-off: building the dictionary costs one full pass, so it pays off for **repeated** lookups. For a single lookup a list is fine. A dictionary also enforces unique keys.

Part 2. Lambdas and delegates

Lambda syntax is the same as TS arrow functions:

```
t => t.Status == TaskItemStatus.Done                // one parameter, expression body
(a, b) => a + b                                     // two parameters
t => { Console.WriteLine(t.Title); return t.Id; }   // block body
```

A lambda needs a **delegate type**. Delegate types are the C# version of function types:

C#	TS	Meaning
`Func<TaskItem, bool>`	`(t: TaskItem) => boolean`	takes TaskItem, returns bool (a predicate)
`Func<int, int, int>`	`(a: number, b: number) => number`	the **last** type argument is the return type
`Func<string>`	`() => string`	no parameters, returns string (used by the demo's `Try` helper)
`Action<string>`	`(s: string) => void`	returns nothing

Part 3. What LINQ actually is: extension methods

LINQ (Language Integrated Query) is a set of **extension methods** on `IEnumerable<T>`, in the static class `System.Linq.Enumerable`. `System.Linq` is imported automatically in .NET projects (implicit usings).

```
// roughly how Where is declared:
public static IEnumerable<T> Where<T>(this IEnumerable<T> source, Func<T, bool> predicate)
//                                    ^^^^ 'this': callable like a method of IEnumerable<T>

tasks.Where(f)            // what you write
Enumerable.Where(tasks, f) // what the compiler turns it into
```

- An **extension method** is a static method in a static class whose first parameter has `this`. It *looks* like an instance method.
- TS analogy: adding a method to `Array.prototype`, but without modifying `Array`, scoped to files that import the namespace, and type-checked.
- Each LINQ operator returns a new `IEnumerable<T>`, so calls **chain**: `.Where(...).Select(...)`, like `.filter().map()`.
- See it yourself: **Cmd+click** on `Where` in Rider opens the `Enumerable` class.

Common mistake

You answered '`.Where` comes from IEnumerable'. Not quite: the `IEnumerable<T>` interface itself has essentially one method, `GetEnumerator()`. `Where` is an **extension method** defined in `System.Linq.Enumerable` and attached to any `IEnumerable<T>` by the `this` parameter.

Part 4. The translation table (the checkpoint)

JS / TS	LINQ	Note
`filter(fn)`	`Where(fn)`	
`map(fn)`	`Select(fn)`	
`flatMap(fn)`	`SelectMany(fn)`	demo 1: all tasks of all projects
`find(fn)`	`FirstOrDefault(fn)`	returns `default(T)` when not found

JS / TS	LINQ	Note
`findIndex(fn)`	`List<T>.FindIndex(fn)`	a List method, not LINQ
`some(fn)`	`Any(fn)`	`Any()` with no args = 'not empty'
`every(fn)`	`All(fn)`	**true on an empty sequence**
`includes(x)`	`Contains(x)`	
`filter(fn).length`	`Count(fn)`	
`reduce(fn, seed)`	`Aggregate(seed, fn)`	prefer `Sum`, `Max`, `Min`, `Average` when they fit
`toSorted(...)`	`OrderBy(key)` / `OrderByDescending(key)` + `ThenBy(key)`	you pass a **key selector**, not a comparator
`slice(a, b)`	`Skip(n).Take(m)`	paging (demo 13)
`[...new Set(xs)]`	`Distinct()`	
`Object.groupBy(xs, fn)`	`GroupBy(fn)`	each group has `.Key` and is itself a sequence
`new Map(...)`	`ToDictionary(k, v)`	**throws** on duplicate keys
`Math.max(...xs.map(fn))`	`Max(fn)`	**throws** on empty sequence of value types
item with the max value	`MaxBy(fn)` / `MinBy(fn)`	returns the whole item
`Array.from(iter)`	`ToList()` / `ToArray()`	runs the query and stores the result

Your translation (quiz)

```
// TS: tasks.filter(t => t.status === 'Done').map(t => t.title)
tasks.Where(t => t.Status == TaskItemStatus.Done)
     .Select(t => t.Title)   // + .ToList() only if you need a stored list

// TS: tasks.find(t => t.id === 3)
tasks.FirstOrDefault(t => t.Id == 3)   // TaskItem? -> null when not found
```

Your insight was right: JS `filter`/`map` are **eager** (they build an array immediately), so `ToList()` is the C# step that 'makes the array'. Small fixes in your version: PascalCase `Title`, and compare `t.Status`, not `t`.

4.1 The find family: choose by intent

Method	0 matches	1 match	2+ matches	Use when
`First`	throws	item	first one	it **must** exist; not existing is a bug
`FirstOrDefault`	`default(T)`	item	first one	it may not exist (like `find`)
`Single`	throws	item	**throws**	exactly one must exist: lookup by unique key (project by `Id`, user by email)
`SingleOrDefault`	`default(T)`	item	**throws**	zero or one; more means a data bug

`Single` **asserts** your data: if two tasks share an Id you get a loud error instead of a silently picked first one.

Gotcha: default is not always null

`FirstOrDefault` returns `default(T)`, which is `null` only for reference types. For a value type it is the zero value: `new List<int> { 5, 7 }.FirstOrDefault(x => x > 100)` returns `0`, and you cannot tell 'not found' from 'found a real 0'. A classic interview gotcha.

4.2 Method syntax vs query syntax

```
// method syntax
allTasks.Where(t => t.Status != TaskItemStatus.Done).OrderBy(t => t.Title).Select(t => t.Title);

// query syntax
from t in allTasks where t.Status != TaskItemStatus.Done orderby t.Title select t.Title;
```

Both compile to the same thing. Most teams use method syntax; recognize query syntax (it looks like SQL).

4.3 Anonymous types

`Select(t => new { t.Id, Upper = t.Title.ToUpper() })` creates objects of a **compiler-generated type with no name** and read-only properties: like returning an object literal from `map`. Because the type has no name, you can only hold it in a `var` inside the method. To return such data from a method or an API, define a **record** (demo section 8: `ProjectSummary`).

Part 5. Deferred execution (the most important idea)

```
IEnumerable<int> query = source.Where(n => n > 1);  // NOTHING runs here: this is a recipe
foreach (var n in query) { }                         // runs NOW
foreach (var n in query) { }                         // runs AGAIN, from scratch
var list = query.ToList();                           // runs once and stores the result
```

- Operators that return `IEnumerable<T>` (`Where`, `Select`, `OrderBy`, `Skip`, `Take`, `GroupBy`, ...) are **deferred**: they describe work.
- Work happens when something **enumerates**: `foreach`, `ToList` / `ToArray` / `ToDictionary`, `Count()`, `First()`, `Any()`, `Sum()`, `string.Join`, ...
- JS analogy: a **generator function**. Calling it does nothing; iterating it runs the body. JS array methods, by contrast, are eager.

Consequences (demo section 11 and your debug session)

1. **The query sees later changes to the source.** `4` was added after the query was defined, and it was included.
2. **Multiple enumeration repeats the work.** 4 items x 2 enumerations = the filter ran **8 times**. With EF Core that is **two SQL round-trips**. Rider warns: 'Possible multiple enumeration'.
3. `ToList()` **freezes a snapshot.** Use it when you need the result more than once, or when later source changes must not affect it.

Common mistake

Quiz: 'why 8 and not 4?' You said 'because we use the query twice'. Half the answer. The other half is **why** that doubles the work: the query stores no results, so each enumeration re-runs the filter over **all 4 items**: 4 x 2 = 8. That is deferred execution.

5.1 Preview: IEnumerable<T> vs IQueryable<T> (topics 10-11)

	`IEnumerable<T>`	`IQueryable<T>`
Name	LINQ to Objects	LINQ to Entities (EF Core)
The lambda is stored as	compiled code	an **expression tree** (data describing the code)
Where the filter runs	in C#, in memory	**in the database**: EF translates it to SQL (`WHERE [Status] = 2`)
Example	`list.Where(t => ...)`	`db.Tasks.Where(t => ...)`

Classic bug: calling `ToList()` too early loads the **whole table** into memory and filters in C#. You answered 'DB' correctly; the type name to remember is `IQueryable<T>`.

Part 6. Gotchas: where C# throws and JS does not

Gotcha	What happens	Fix
`First(...)` with no match	`InvalidOperationException: Sequence contains no matching element`	`FirstOrDefault` + null check, if 'not found' is valid
`Single(...)` with 2+ matches	`InvalidOperationException: ... more than one matching element`	use `Single` only for unique keys
`Max()`/`Min()`/`Average()` on empty sequence of value types	`InvalidOperationException: Sequence contains no elements` (JS `Math.max()` gives `-Infinity`)	check `Any()` first, or `Max(t => (DateTime?)t.CreatedAt)` returns null
`All(...)` on empty sequence	returns `true` (same as JS `[].every`)	decide whether empty counts as success
`dict[missingKey]`	`KeyNotFoundException`	`TryGetValue(key, out var value)`
`ToDictionary` with duplicate keys	`ArgumentException: An item with the same key has already been added`	unique keys, or `GroupBy` / `ToLookup`
Multiple enumeration	work (or SQL) runs again	`ToList()` once and reuse
Changing a List inside its own `foreach`	`InvalidOperationException: Collection was modified`	iterate a copy (`ToList()`) or apply changes after the loop

Part 7. var: when and why (your question)

You asked why the demo sometimes uses `var` and sometimes an explicit type, e.g.:

```
var shortInfo = allTasks.Select(t => new { t.Id, Upper = t.Title.ToUpper() }).Take(2);
TaskItem? found = allTasks.FirstOrDefault(t => t.Id == 5);
```

What `var` is: compile-time type inference, like TS `const x = 5`. The variable is still **statically typed**: `var x = 5;` is an `int` forever, and `x = "hello"` is a compile error. It is not `any` and not JS `var`. (C# has a separate `dynamic` keyword that disables type checking; you will almost never use it.)

Case	Example	Why
`var` **required**: anonymous types	`var shortInfo = ... new { t.Id, ... }`	the type has no name, so there is nothing to write
Type **required**: no initializer or `null`	`TaskItem? current = null;`	nothing to infer from
Type **required**: collection expression	`List<int> ids = [];`	`var ids = [];` does not compile: `[]` needs a target type (List? array? HashSet? which element type?)
Type **required**: you want a different type	`IEnumerable<TaskItem> items = list;`	e.g. expose as read-only; `double d = 5;`
Style choice	`TaskItem? found = ...` vs `var found = ...`	identical result; the demo writes types to **teach what each LINQ method returns**

Team rule (Microsoft style guide, Rider default): `var` when the type is obvious from the right side (`var p = new Project { ... }`); explicit type when it helps the reader (`Dictionary<int, Project> byId = projects.ToDictionary(p => p.Id);`). Teams often enforce it in `.editorconfig`. The reverse shortcut also exists: **target-typed new**, `List<int> ids = new();`.

Common mistake

'Why must `shortInfo` use `var`?' You answered 'because its type comes from the method that returns the value'. That is true of **every** `var`. The real reason: an anonymous type has **no name** you could write. 'Does `var x = 5` make x dynamically typed?' You said 'yes, it's int; can't assign hello'. The facts were right but the first word was wrong: the answer is **No**, it is statically typed.

Part 8. Our code, explained

8.1 src/ProjectHub.Api/Data/SampleData.cs

Shortened for print (the real file lists tasks 5-7: Login screen, Push notifications, App store listing).

```
using ProjectHub.Api.Entities;

namespace ProjectHub.Api.Data;

public static class SampleData
{
    // Returns a NEW list every call, so one caller's changes never leak into another's data.
    public static List<Project> CreateProjects()
    {
        var day = new DateTime(2026, 9, 1, 9, 0, 0, DateTimeKind.Utc);

        return
        [
            new Project
            {
                Id = 1, Name = "Website redesign", Description = "New marketing site", CreatedAt = day,
                Tasks =
                [
                    new TaskItem { Id = 1, ProjectId = 1, Title = "Design mockups",
                                   Status = TaskItemStatus.Done, CreatedAt = day.AddDays(1) },
                    new TaskItem { Id = 2, ProjectId = 1, Title = "Build landing page",
                                   Status = TaskItemStatus.InProgress, CreatedAt = day.AddDays(3) },
                    new TaskItem { Id = 3, ProjectId = 1, Title = "SEO audit", CreatedAt = day.AddDays(5) },
                    new TaskItem { Id = 4, ProjectId = 1, Title = "Write copy",
                                   Status = TaskItemStatus.Done, CreatedAt = day.AddDays(2) },
                ],
            },
            new Project { Id = 2, Name = "Mobile app", CreatedAt = day.AddDays(4), Tasks = [ /* tasks 5-7 */ ] },
            new Project { Id = 3, Name = "Internal tools", Description = "Admin dashboards",
                          CreatedAt = day.AddDays(8) },   // no tasks on purpose
        ];
    }
}
```

Decision	Why
`public static class`	Cannot be instantiated, holds only static members. Like a TS module exporting functions.
A method returning a **new** list each call (not a `static List` field)	A shared static list is **global mutable state**: one caller's `Add` would be visible to everyone; in a web server, to every request and thread at once. A factory gives each caller its own copy.
Fixed dates with `DateTimeKind.Utc`	Deterministic output (same every run) and consistent with `UtcNow` from topic 1.
Collection expressions `[ ... ]`	The target type (`List<Project>`, `List<TaskItem>`) tells the compiler what to build.
Setting `CreatedAt` although it is `init`	Allowed: an object initializer **is** the moment of creation.
Some tasks without `Status`	They get the default `Todo` from the entity's initializer.
Project 3 has **no tasks**	Empty collections are where the `All`, `Max`, `Average` gotchas show up.
Lives in `Data/` of the API project	Topic 6 uses it to fill the in-memory storage behind the first endpoints.

8.2 playground/02-linq.cs: how to open and run it

1. Get it on your Mac: `git fetch origin`, `git checkout claude/practical-bohr-1b5pb9`, `git pull` (or Rider: **Git > Pull**).
2. Find it in Rider: the **Solution** view shows only what is listed in `ProjectHub.slnx`. A `playground` solution folder was added, so it appears above `src`. Alternative: switch the panel from **Solution** to **File System** view, which shows the real folders.

3 Run it: open Rider's **Terminal** tab (it starts in the repo root) and run `dotnet run playground/02-linq.cs`.

The first line: #:project

```
#:project ../src/ProjectHub.Api/ProjectHub.Api.csproj
```

The demo is a single standalone file (a .NET 10 'file-based app'), not part of any project. Normally it could not see `Project`, `TaskItem` or `SampleData`. This directive tells `dotnet run`: build the API project at this path (relative to the file: `..` goes up from `playground/`) and let me use its classes. TS analogy: `import { Project } from '../src/entities'` in a script. `#:` lines exist only in single-file programs; recognize them, you will not write them in project code.

Other C# features used in the demo

Code	Meaning
`record ProjectSummary(int Id, string Name, int TaskCount, int DoneCount);`	a DTO-shaped result: projection entity -> summary (topic 1: records for data crossing a boundary)
`new ProjectSummary(p.Id, p.Name, TaskCount: p.Tasks.Count, ...)`	**named arguments**: optional labels for readability
`static void Section(string title) => ...`	local helper function; `static` = cannot capture outer variables
`static void Try(string label, Func<string> action)`	takes a function `() => string` and runs it inside `try/catch` to show exceptions without crashing
`foreach (var (status, count) in countByStatus)`	**deconstruction** of each `KeyValuePair`, like `for (const [k, v] of map)`
`$"{t.Status,-10} {t.CreatedAt:yyyy-MM-dd}"`	interpolated string with **alignment** (`,-10` = pad to 10, left) and **format** (`:yyyy-MM-dd`)
`projectsById.TryGetValue(42, out Project? p42)`	**out parameter**: the method returns bool and writes the found value into `p42`

8.3 The 13 sections at a glance

#	Section	Shows
1	SelectMany	flatten tasks of all projects (7 tasks in 3 projects)
2	Where	filter InProgress tasks
3	Select	map to titles; anonymous type projection
4	OrderBy / ThenByDescending	sort by status, then newest first; source untouched
5	First / FirstOrDefault / Single	find; First and Single throwing
6	Any / All / Count / Contains	some / every / length / includes; All on empty = true
7	GroupBy	groups with `.Key`, each group is a sequence
8	Projection into a record	entity -> `ProjectSummary` DTO
9	Sum / Max / Average / MaxBy / Aggregate	aggregates; Max on empty throws
10	Dictionary	ToDictionary, indexer, TryGetValue, KeyNotFoundException, GroupBy -> dictionary
11	Deferred execution	query sees later changes, runs twice, ToList snapshot

#	Section	Shows
12	Query syntax	SQL-like spelling of the same query
13	Skip / Take	paging, like `slice`

Full output (captured in the cloud container)

```
=== 1. SelectMany: flatten all tasks of all projects ===
7 tasks in 3 projects

=== 2. Where: filter ===
#2 Build landing page
#5 Login screen

=== 3. Select: map / projection ===
Design mockups, Build landing page, SEO audit, Write copy, Login screen, Push notifications,
    App store listing
{ Id = 1, Upper = DESIGN MOCKUPS }
{ Id = 2, Upper = BUILD LANDING PAGE }

=== 4. OrderBy / ThenBy: sort (returns a NEW sequence, source untouched) ===
Todo       2026-09-08 App store listing
Todo       2026-09-07 Push notifications
Todo       2026-09-06 SEO audit
InProgress 2026-09-05 Login screen
InProgress 2026-09-04 Build landing page
Done       2026-09-03 Write copy
Done       2026-09-02 Design mockups

=== 5. First / FirstOrDefault / Single: find ===
FirstOrDefault(Id 5):  Login screen
FirstOrDefault(Id 99): null
First(Id 99): THROWS InvalidOperationException: Sequence contains no matching element
Single(Status Done): THROWS InvalidOperationException: Sequence contains more than one
    matching element

=== 6. Any / All / Count / Contains: some / every / length / includes ===
Any task Done?              True
All tasks have a title?     True
Count of Todo:              3
Titles contain 'SEO audit'? True
Empty project: all tasks Done? True

=== 7. GroupBy: group tasks by status ===
Done: 2 -> Design mockups, Write copy
InProgress: 2 -> Build landing page, Login screen
Todo: 3 -> SEO audit, Push notifications, App store listing

=== 8. Projection into a record (a DTO shape) ===
ProjectSummary { Id = 1, Name = Website redesign, TaskCount = 4, DoneCount = 2 }
ProjectSummary { Id = 2, Name = Mobile app, TaskCount = 3, DoneCount = 0 }
```

```
ProjectSummary { Id = 3, Name = Internal tools, TaskCount = 0, DoneCount = 0 }

=== 9. Aggregates: Sum / Max / Average / MaxBy / Aggregate (reduce) ===
Total tasks (Sum):   7
Newest task (Max):   2026-09-08
Avg tasks/project:   2.33
MaxBy (whole item):  App store listing
Aggregate (reduce):  Design mockups | Build landing page | SEO audit | Write copy | Login
    screen | Push notifications | App store listing
Max on empty project: THROWS InvalidOperationException: Sequence contains no elements

=== 10. Dictionary<TKey, TValue>: fast lookup by key (TS: Map) ===
projectsById[2].Name = Mobile app
TryGetValue(42): not found
projectsById[42]: THROWS KeyNotFoundException: The given key '42' was not present in the
    dictionary.
Done = 2
InProgress = 2
Todo = 3

=== 11. Deferred execution: a query is a RECIPE, not a result ===
After defining the query: evaluations = 0  (nothing ran yet)
First enumeration:  [2, 3, 4]   (sees the added 4!)
Second enumeration: [2, 3, 4]   (runs the filter AGAIN)
evaluations = 8  (4 items x 2 enumerations)
snapshot after adding 5: [2, 3, 4]  (frozen)

=== 12. Query syntax (same thing, SQL-like spelling) ===
App store listing, Build landing page, Login screen, Push notifications, SEO audit

=== 13. Paging: Skip / Take (what an API list endpoint does) ===
Page 1: #4, #5, #6
```

Part 9. Debugging: deferred execution live (Rider)

Goal: see with the debugger that the lambda inside `Where` runs only when the query is read, and runs again on every read. We used the proven approach from topic 1: a temporary snippet in `Program.cs`.

9.1 Steps

1. Paste the snippet below into `src/ProjectHub.Api/Program.cs`, right before `app.Run();`.
2. Set two breakpoints (click the gutter): **A** on `evaluations++;`, **B** on `var first = query.ToList();`.
3. Start Debug (bug icon).
4. First stop: note which breakpoint and the value of `evaluations` in Variables.
5. Click **Resume** (green play). At A: read `n`; look at the **Frames** list (left side of the Threads & Variables tab).
6. Keep resuming, count the hits of A and note each `n`. Right-click `evaluations` > **Add to Watches** to keep it visible.
7. Keep resuming until `ToDictionary` throws; Rider stops and shows the exception.
8. Clean up: delete the snippet and the breakpoints.

```
var source = new List<int> { 1, 2, 3 };
int evaluations = 0;
IEnumerable<int> query = source.Where(n =>
{
    evaluations++;              // BREAKPOINT A
    return n > 1;
});
source.Add(4);
var first = query.ToList();     // BREAKPOINT B
var second = query.ToList();
var dict = new[] { 1, 1 }.ToDictionary(x => x);   // breaks on purpose
```

9.2 What you observed

Question	Your answer	What it proves
A or B first? `evaluations`?	B, `evaluations` = 0 (correct)	Defining the query ran **nothing**: deferred execution.
How many A hits? Which `n`?	8 times: 1, 2, 3, 4 twice (correct)	The query includes `4` (added later) and re-runs for every enumeration.
What did `ToDictionary` throw?	`ArgumentException: An item with the same key has already been added. Key: 1`, because `[1, 1]` has a duplicate (correct)	Dictionary keys must be unique.
Which of your lines was in Frames below the lambda?	'what?' (see 9.3)	The lambda was called from the `ToList()` line, not the definition line.

9.3 Frames = the call stack (your question)

Frames is the **call stack**: the chain of function calls that led to the point where the program is paused. You know it from Chrome DevTools (Sources tab > Call Stack panel when paused in JS) and from error stack traces. The exception text you pasted is a printed call stack:

```
at ThrowAddingDuplicateWithKeyArgumentException   <- where it blew up (top)
at ... ToDictionary ...                           <- who called that
at Program.<Main>$ ... Program.cs:line 28         <- YOUR line that started it (bottom)
```

While paused at breakpoint A (inside the lambda), the stack looked like this:

Frame	What it is
lambda `(n) => { evaluations++; ... }`	top: where you are paused
`Where...Iterator` / `ToList` internals	.NET LINQ code calling your lambda
Program.cs: `var first = query.ToList();`	**your line** that triggered the call

Remember

The call stack itself proves deferred execution: the lambda was called **from the** `ToList()` **line**, not from the line that defined `query`. Clicking a frame in Rider jumps to that line: the main tool for 'how did I get here?' (heavily used in topic 5).

Debug vocabulary so far: **breakpoint**, **Step Over**, **Resume**, **Variables**, **Watches** (pinned expressions), **Frames / call stack**, **exception break**.

Part 10. Questions, corrections, interview Q&A

10.1 Questions you asked

Q: How do I open `playground/02-linq.cs`? Where is it?

It is on GitHub on the working branch; pull it to your Mac first. Rider's Solution view shows only solution items, so a `playground` folder was added to `ProjectHub.slnx`; alternatively switch to File System view. Run it from Rider's Terminal: `dotnet run playground/02-linq.cs`.

Q: What does `#:project ../src/ProjectHub.Api/ProjectHub.Api.csproj` mean?

A .NET 10 single-file directive: build that project and let this standalone file use its classes. Like importing from a workspace package in a TS script.

Q: Why sometimes `var` and sometimes not?

See Part 7: required for anonymous types; explicit types required for null/no initializer, `[]`, or a different target type; otherwise style.

Q: Why can't I read the lesson .md file inside Claude?

The Claude app shows Markdown as raw text. Lessons are now also rendered to formatted HTML in `docs/lessons/html/` (`scripts/lessons_to_html.py`); in Rider, the Markdown **Preview** toggle also works. CLAUDE.md now says: always send the rendered version.

Q: What is 'Frames' in the debugger?

The call stack (Part 9.3).

10.2 Mistakes we corrected

You said	Correct
`.Where(t => t == ...).Select(t => t.title)`	`.Where(t => t.Status == TaskItemStatus.Done).Select(t => t.Title)`: compare the property, PascalCase names.
Filter ran 8 times 'because we use the query twice'	...and each enumeration re-runs the filter over all 4 items (deferred execution): 4 x 2 = 8.
`shortInfo` needs `var` because its type comes from the method	Every `var` infers from the right side. Anonymous types need `var` because they have **no name**.
`var x = 5` is dynamically typed: 'yes'	**No**: statically typed `int`; `x = "hello"` is a compile error.
`var ids = []` fails 'because it is not typed'	Precisely: `[]` needs a **target type** to know which collection and element type to build.
`.Where` comes from IEnumerable	It is an **extension method** in `System.Linq.Enumerable` (`this IEnumerable<T>` parameter).
'Dictionary, of course' (no reason)	O(1) hash lookup vs O(n) scan; pays off for repeated lookups.
Single example skipped	Lookup by unique key: project by `Id`, user by email. Throws on duplicates.

10.3 Interview questions with model answers

Q: Translate map / filter / find / some / every / flatMap / reduce into LINQ.

`Select`, `Where`, `FirstOrDefault`, `Any`, `All`, `SelectMany`, `Aggregate` (or `Sum`/`Max`/... when they fit).

Q: `First` vs `FirstOrDefault` vs `Single` vs `SingleOrDefault`?

First: must exist, throws on 0. FirstOrDefault: may not exist, returns default(T). Single: exactly one, throws on 0 or 2+. SingleOrDefault: 0 or 1, throws on 2+. Single asserts uniqueness, e.g. lookup by Id.

Q: What does `FirstOrDefault` return for `List<int>` when nothing matches?

`0` (default of int), not null. Ambiguous with a real 0.

Q: What is deferred execution? Name two consequences.

LINQ operators returning IEnumerable describe work; it runs when enumerated. Consequences: the query sees later source changes; enumerating twice runs the work twice (twice the SQL with EF).

Q: When and why call `ToList()`?

When the result is used more than once, when you need a stable snapshot, or to materialize before leaving a scope (e.g. a DbContext). Not too early with EF: it loads data into memory.

Q: `List<T>` or `IEnumerable<T>` as a parameter?

IEnumerable (or IReadOnlyCollection/IReadOnlyList if you need Count/index): accept the most general type. Return a specific, useful type.

Q: Dictionary vs List for lookups by id?

Dictionary: O(1) hash lookup vs O(n) scan; worth it for repeated lookups; also enforces unique keys.

Q: `list.Sort()` vs `OrderBy`?

Sort mutates the list in place (like JS sort). OrderBy returns a new lazy sorted sequence; the source is untouched (like toSorted / lodash orderBy).

Q: What is an extension method?

A static method in a static class with `this` on the first parameter; callable like an instance method. LINQ's `Where`, `Select`, etc. are extension methods on IEnumerable<T>.

Q: `IEnumerable<T>` vs `IQueryable<T>`?

IEnumerable: LINQ to Objects, lambda is compiled code, runs in memory. IQueryable: lambda is an expression tree, EF translates it to SQL, runs in the database.

Q: What does `var` do? Is it dynamic typing?

Compile-time type inference; the variable is statically typed. Required for anonymous types. Not `dynamic`.

Q: What is `Func<T, bool>`? `Action<T>`?

Delegate types: a function taking T returning bool (predicate); a function taking T returning nothing.

Q: Why might `Max()` throw?

On an empty sequence of a value type: InvalidOperationException. Check Any() or project to a nullable.

Q: Why should sample/shared data not be a static List?

Global mutable state: shared across callers, requests and threads; one caller's change leaks to all.

Q: What is the call stack / Frames panel?

The chain of calls that led to the current point; read bottom-up to find your line that started it.

Part 11. Summary

Key takeaways

Collections: `List<T>` (array), `Dictionary<K,V>` (Map, O(1) lookup, indexer throws on missing key), `HashSet<T>` (Set), `IEnumerable<T>` (Iterable).
Accept general types (`IEnumerable<T>`), return specific ones.
Lambdas look like arrow functions; their types are delegates: `Func<..., TResult>`, `Action<...>`.
LINQ = extension methods on `IEnumerable<T>` in `System.Linq.Enumerable`. They chain like `.filter().map()`.
Find family by intent: `First` (must exist), `FirstOrDefault` (may not exist, returns default(T)), `Single` (exactly one, asserts uniqueness).
Deferred execution: a query is a recipe; it runs when enumerated and re-runs every time. `ToList()` runs once and snapshots.
`IQueryable<T>` (EF Core) turns the same LINQ into SQL that runs in the database.
C# throws where JS returns undefined/-Infinity: First, Single, Max on empty, dict[missing], ToDictionary with duplicates.
`var` = static type inference; required for anonymous types; `[]` needs a target type.
Debugger: Frames = call stack; it showed the lambda being called from the `ToList()` line.

Cheat sheet

JS / TS	C#
`arr.filter(f).map(g)`	`list.Where(f).Select(g)`
`arr.find(f)`	`list.FirstOrDefault(f)`
`arr.some(f)` / `arr.every(f)`	`list.Any(f)` / `list.All(f)`
`arr.flatMap(f)`	`list.SelectMany(f)`
`arr.reduce(f, seed)`	`list.Aggregate(seed, f)`
`[...arr].sort(...)`	`list.OrderBy(x => key).ThenBy(...)`
`arr.slice(a, a + n)`	`list.Skip(a).Take(n)`
`new Map(...)` / `map.get(k)`	`ToDictionary(...)` / `dict.TryGetValue(k, out var v)`
`new Set(arr)`	`new HashSet<T>(list)` or `Distinct()`
`Array.from(gen())`	`query.ToList()`
`(t: T) => boolean`	`Func<T, bool>`
generator function (lazy)	`IEnumerable<T>` query (deferred)

Checklist: can you do this without notes?

- Translate any JS array chain into LINQ, and say whether it runs eagerly or lazily.
- Explain the find family and give an example for `Single`.
- Explain deferred execution and why the demo's filter ran 8 times.
- Explain when to call `ToList()`, and the danger of calling it too early with EF Core.
- Explain `IEnumerable` vs `IQueryable`.
- Explain what an extension method is and how `.Where` reaches a `List<T>`.
- Explain Dictionary vs List lookups with Big-O.

- Explain `var` and when it is required.
- Read a call stack (Frames) and find your own line in it.

What comes next: Topic 3 (Interfaces, generics, exceptions)

You already met interfaces (`IEnumerable<T>`) and generics (`List<T>`, `Func<T, bool>`) from the user side. Topic 3 defines our own: `IProjectService` + `ProjectService` (which will use LINQ over `SampleData`), one business validation rule, and exceptions: throwing, catching, and when an error is 'expected'. Checkpoint: explain interface vs class; handle an expected error.